

Real-Time Bidirectional Sign Language Translator: Sign-to-Speech, Speech-to-Sign, Networked Live Translation, and Web-Based Interface

Shashank Singh

Department of Data Science,
Mukesh Patel School of
Technology
Management & Engineering
SVKM's NMIMS, Mumbai, India
shashank.singh020@nmims.in

Bhavya Sharma

Department of Data Science,
Mukesh Patel School of
Technology
Management & Engineering
SVKM's NMIMS, Mumbai, India
bhavya.sharma010@nmims.in

Archana Lakhe

Department of Data Science,
Mukesh Patel School of
Technology
Management & Engineering
SVKM's NMIMS, Mumbai, India
Archana.Lakhe@nmims.edu

Abstract — *One of the most persistent accessibility challenges in modern society remains to be the issue of Communication between the hearing-impaired and the general hearing population. Nearly, over 466 million people around the globe live with disabling hearing loss (WHO, 2023), yet only fewer than 2% of the hearing population can fully communicate in sign language. The existing technological solutions are predominantly unidirectional where none of them provide a real-time, offline, and bidirectional system capable of running on commodity hardware with integrated NLP and networked peer-to-peer capability. This paper presents a Real-Time Bidirectional Sign Language Translator comprising of two simultaneously operating modules which are: 1) Module 1 (Sign-to-Speech) uses Google MediaPipe Holistic for hand landmark extraction, a 2-layer LSTM trained on a personal webcam dataset, and a T5-small NLP model for converting fingerspelled letters into grammatical English sentences spoken via TTS and; 2) Module 2 (Speech-to-Sign) transcribes spoken language via Google STT and displays ASL GIF animations. The key architectural contributions include: (1) removal of pose and face landmarks reducing dimensionality from 1,662 to 126, (2) wrist-origin scale normalization achieving position and distance invariance, (3) a motion gate and confidence gap mechanism for near-zero false positives, and (4) a TCP peer-to-peer networked live translation mode enabling simultaneous bidirectional ASL communication between two machines—a capability not previously demonstrated in published literature. A web interface with WebSocket connection streaming across the system which achieved est accuracy of 98.96% with 32 ASL classes on a CPU hardware.*

Keywords — Sign Language Recognition, LSTM, Bidirectional Translation, MediaPipe, T5 Gloss, Real-Time Inference

I. INTRODUCTION

A. Background and Motivation

The primary mode of exchanging conversation for the hearing-impaired is known as Sign Language. There are more than 300 different sign languages used across the world which includes American Sign Language (ASL), Indian Sign Language (ISL), etc. Despite their abundance and importance, sign languages remain inaccessible to most of the hearing

population, creating a communication barrier that affects education, healthcare, employment, and daily social interactions.

The motto of this project can be perfectly captured by the analogy of live concert sign language interpreters—these are the professionals who simultaneously translate spoken performance into sign language in real time for hearing-impaired people. This project automates that experience, enabling a deaf person and a hearing person to hold a natural conversation without a human interpreter, over a live connection.

B. Problem Statement

Current sign language recognition system suffers from four critical limitations: (1) **Unidirectionality**- most systems convert sign to text or vice-versa, but not both; (2) **GPU dependency**-high-accuracy approaches require dedicated GPU hardware; (3) **Domain gap**-models fail when signer distance or position changes; (4) **No NLP integration**- raw sign labels output without grammatical correction or sentence formation.

C. Objectives

The objectives include (1) develop real-time sign-to-speech achieving >95% accuracy on ASL using only a webcam; (2) develop simultaneous speech-to-sign display; (3) eliminate domain gap through wrist-origin normalization; (4) integrate T5-small NLP for sentence formation; (5) implement TCP peer-to-peer networked live translation; (6) build a production web interface for browser-based operation.

D. Scope

The system covers 32 ASL classes: 26 alphabet letters (A–Z) and 6 gesture signs (Hello, Thanks, Yes, No, Sorry, I Love You). The networked mode operates over local WiFi. The web interface (SignBridge) runs on any modern browser via Flask-SocketIO.

II. LITERATURE REVIEW

A comprehensive review of 20 Scopus and IEEE indexed papers was conducted. Four key themes have emerged: (1) landmark-based LSTM outperforms image CNN for real-time recognition; (2) domain gap is the primary deployment challenge; (3) no truly bidirectional offline system existed prior to this work; (4) NLP post-processing is absent from all reviewed systems.

A. Landmark-Based Recognition

Sundar and Bagyammal (2022) demonstrated MediaPipe + LSTM achieves 84.2% on ASL alphabets, validating the landmark-based approach. Khartheesvar et al. (2024) used full Holistic features (1,662-dim) and found face landmarks introduced confusion between signs performed near the face—directly motivating our landmark reduction strategy. Farhan and Madi (2022) correlated hand detection rates with accuracy, informing our hand-presence validation gate.

B. Inference Stabilization

Gaikwad and Admuthe (2024) introduced stability buffers showing a 5-frame buffer reduced false positives by 73%. Dardas and Georganas (2011) established 5-frame temporal smoothing eliminates 89% of single-frame noise. Mercanoglu et al. (2019) confirmed 30 frames at 30fps as the optimal choice, backing our sequence length choice.

C. Domain Gap and Normalization

Zhang and Jiang (2024) surveyed 200+ SLR papers identifying domain gap as the primary unsolved deployment challenge, recommending architectural invariance over data augmentation. Bohacek and Hruz (2022) demonstrated body-center normalization improved cross-signer accuracy from 71% to 84%, which we extended to wrist-origin normalization achieving both position and scale invariance.

D. Similar Sign Disambiguation

Kothadiya et al. (2022) identified attention mechanisms as key for distinguishing visually similar signs differing only in orientation. Pathan et al. (2023) showed image-landmark fusion improved accuracy by 6.3% on ambiguous pairs. KhajehPourian and Zarei (2024) found fingertip positions and inter-finger gaps are the most discriminative features—validated by our cosine angle discriminator results.

E. Bidirectional Systems and NLP

Anonymous (2024) presented the first published bidirectional system using MobileNetV2 + Blender 3D animations, achieving 80.5% but requiring GPU and lacking NLP integration. Zhou et al. (2021) demonstrated SignBERT's contextual attention, motivating our T5-small integration as a lightweight NLP post-processing layer without requiring sign-specific pretraining.

F. Research Gaps

TABLE I: RESEARCH GAPS AND PROPOSED SOLUTIONS

Research Gap	Our Solution
Unidirectional only	Simultaneous bidirectional modules
Domain gap — scale sensitivity	Wrist-origin normalization (126-dim)
No NLP sentence formation	T5-small gloss conversion module
False positives in transitions	Motion gate + confidence gap
GPU dependency	CPU-only, 15ms inference on M1
No networked translation	TCP peer-to-peer live translator
No browser interface	SignBridge web app (Flask-SocketIO)

III. METHODOLOGY

A. Dataset Collection

A personal signer webcam dataset was collected using a custom SPACE-triggered recording script. Signer design eliminates intra-class variance from multiple signers, bridging the domain gap identified by Zhang and Jiang (2024). Specifications: 32 ASL classes, 30

sequences per class, 30 frames per sequence at 30fps, which sums up to 960 sequences. Captured on personal device (Macbook Air) with built-in webcam and consistent lighting and fixed camera position. Hand presence ensured 100% clean data.

B. Feature Engineering

1) Initial Feature Space:

MediaPipe Holistic provides 1,662-dim features: 468 face landmarks $\times$ 3 (1,404-dim), 33 pose landmarks $\times$ 4 (132-dim), and 21 hand landmarks $\times$ 3 $\times$ 2 hands (126-dim). Face and pose represent 92.4% of the raw vector.

2) Pose and Face Removal:

Following findings of Khartheesvar et al. (2024) and Zhang and Jiang (2024), all face and pose landmarks were discarded, only keeping the 126-dim hand keypoints. This resolved I LOVE YOU/Thanks confusion and eliminated pose induced domain gap.

3) Wrist-Origin Normalization:

Two-step normalization: (a) **Translation**—subtract wrist landmark (index 0) from all 21 landmarks, centering at origin; (b) **Scale normalization** is divided by maximum Euclidean distance from wrist to any landmark. The results were 126-dim vector invariant to signer position and distance from the camera. The same hand configuration produces similar features at 30cm or 100cm from the camera.

C. Model Architecture

Training configuration: Adam (lr=0.0005), categorical cross-entropy, balanced class weights via `compute_class_weight`, EarlyStopping (patience=50, `restore_best_weights=True`), batch=16, val_split=10%, max_epochs=500. Five independent training runs were conducted with best model which was selected on the basis of val_loss, reducing the accuracy variance from ~3% to <0.5%.

TABLE II: LSTM MODEL ARCHITECTURE

Layer	Output Shape	Parameters
Input	(None, 30, 126)	—
LSTM (return sequences)	(None, 30, 128)	131,584
LSTM	(None, 64)	49,408
Dense (ReLU)	(None, 64)	4,160
Dense (ReLU)	(None, 32)	2,080
Dense (Softmax)	(None, 32)	1,056
Total	—	188,288

D. Inference Pipeline

Four Gates ensure high-precision predictions: (1) **Hand Presence Gate** checks MediaPipe must detect at least one hand; (2) **Motion Gate** ensures standard deviation of keypoints across last 6 frames which must be below 0.012, preventing prediction during transitions; (3)

Confidence Threshold must be 75% for gesture signs and 92% for letters; (4) **Confidence Gap** ensures top prediction must exceed second by ≥ 25 percentage points. A ten-frame stability buffer requires unanimous agreement before sign acceptance. A three second cooldown prevents duplicate detections.

E. Cosine Angle Discriminator

For visually identical sign pairs, cosine angle between the normalized hand vector and precomputed class mean vectors provides clarification. The discriminator is invoked when confidence is below 99% or when the confidence gap is below 50%. All 6 primary confused pairs confirmed separable:

TABLE III: ANGULAR SEPARATION OF CONFUSED SIGN PAIRS

Sign A	Sign B	Angle Gap
O	E	6.93°
Yes	I Love You	5.58°
V	Z	22.60°
N	J	12.43°
B	C	5.22°
V	W	6.71°

F. T5 Gloss Module

Fingerspelling individual letters produces isolated characters without semantic meaning. The T5 Gloss module (gloss_t5.py) converts accumulated sign buffers into grammatical English through three stages: (1) **Letter Grouping**—consecutive letter signs joined into words (H,E,L,L,O → HELLO); (2) **Rule Fast Path**—known gesture combinations resolved instantly (HELLO + THANKS → 'Hello, thank you!'); (3) **T5-small Generation**—unknown combinations passed to T5-small with prompt 'convert ASL gloss to English sentence: [TOKENS]', beam search (num_beams=4). The module is triggered by ENTER key press.

G. Speech-to-Sign Module

Module 2 (Speech TO Sign.py) uses PyAudio for push-to-talk recording—eliminating silence-detection latency of standard speech_recognition.listen(). SPACE key starts recording, SPACE again stops and immediately triggers Google STT transcription. Each word is looked up in the ASL GIF library; unknown words are fingerspelled letter-by-letter as fallback.

H. Networked Live Translation

live_translator.py establishes a peer-to-peer TCP socket connection (port 9999) between two machines on the same network. Both machines run identical scripts—one as HOST, one as CLIENT. Each machine runs four concurrent threads: (1) Sign Detection Thread: webcam → LSTM → sign buffer → send sentence to peer; (2) GIF Display Thread: receive sign sentences → play ASL GIFs; (3) Speech Input Thread: push-to-talk mic → STT → send transcript to peer; (4) TTS Receiver Thread: receive speech text → TTS speak + queue GIFs.

A split-screen OpenCV window shows the signer's webcam feed on the left and peer GIFs on the right. This architecture enables simultaneous bidirectional translation with no turn-taking—the first such networked ASL system demonstrated in published literature, analogous to a sign language video call.

I. SignBridge Web Interface

A production-grade web application (SignBridge) was developed using Flask-SocketIO as the backend bridge between the React-based frontend and the ML pipeline. The interface features four tabs: (1) **Sign to Speech**—live webcam feed streamed via base64-encoded JPEG WebSocket frames, real-time confidence bars, top-3 prediction display, sign buffer chips, and T5 gloss conversion; (2) **Speech to Sign**—Web Speech API push-to-talk with continuous recognition, real-time interim transcript display, and ASL GIF queue playback; (3) **Live Network**—host/client configuration panel for live_translator.py sessions; (4) **History**—persistent conversation log with export capability. The backend (app.py) runs the full sign detection loop on a daemon thread, encoding OpenCV frames to JPEG and emitting them via socketio at ~30fps.

IV. SYSTEM DESIGN AND ARCHITECTURE

A. Overall Architecture

The system consists of two primary modules working in parallel, connected by a shared display layer and an optional network transport layer. Module 1 pipeline: Webcam → MediaPipe Holistic → Wrist-Normalized 126-dim → LSTM (98.96%) → Stability Buffer → T5 Gloss → TTS. Module 2 pipeline: Microphone (Push-to-Talk) → PyAudio → Google STT → Word Lookup → ASL GIF Display / Fingerspelling.

B. Technology Stack

TABLE IV: SYSTEM TECHNOLOGY STACK

Component	Technology	Purpose
Hand Tracking	MediaPipe Holistic 0.10.14	21-landmark hand detection
ML Framework	TensorFlow 2.16.2 / Keras	LSTM training and inference
Vision	OpenCV 4.x	Webcam capture, display, UI
NLP	HuggingFace T5-small	Gloss-to-English conversion
Speech Input	PyAudio + SpeechRecognition	Push-to-talk + Google STT
Speech Output	pyttsx3	Offline TTS synthesis
Networking	Python socket (TCP)	Peer-to-peer translation
Web Backend	Flask + Flask-SocketIO	WebSocket ML bridge
Web Frontend	HTML/CSS/JS (SignBridge)	Browser-based interface
Hardware	MacBook Air M1	CPU-only inference platform

V. RESULTS AND ANALYSIS

A. Model Accuracy

The normalized 126-dim hand-only LSTM model achieved 98.96% test accuracy across 32 classes on a held-out test split of 96 sequences (10% stratified split). This represents the best result across 5 independent training runs. 31 of 32 classes achieved perfect precision and recall. Sign J (0.667 recall) is the only exception due to its motion trajectory nature.

B. Per-Class Performance (Selected)

TABLE V: PER-CLASS PERFORMANCE (SELECTED)

Class	Precision	Recall	F1	Note
Hello	1.000	1.000	1.000	
Thanks	1.000	1.000	1.000	
Yes	1.000	1.000	1.000	
I Love You	1.000	1.000	1.000	
A–I	1.000	1.000	1.000	
J	1.000	0.667	0.800	Motion sign
K–Z (excl. J)	1.000	1.000	1.000	

C. Comparison with Related Work

TABLE VI: COMPARISON WITH EXISTING SYSTEMS

System	Acc.	Bidir.	CPU	NLP	Web
Sundar and Bagyammal (2022)	84.2%	No	Yes	No	No
Khartheesvar et al. (2024)	87.4%	No	Yes	No	No
Anonymous (2024)	80.5%	Yes	No (GPU)	No	No
Gaikwad and Admuthe (2024)	100%*	No	Yes	No	No
Our System (2026)	98.96%	Yes	Yes	Yes	Yes

* 9-class limited vocabulary only.

D. Inference Latency

On MacBook Air M1 (CPU only), average per-frame inference latency: ~15ms, enabling true 30fps real-time operation. This is 12× faster than the 180ms GPU latency reported by Huang et al. (2024) for video-based R(2+1)D+LSTM, while achieving comparable or superior accuracy.

E. Web Interface Performance

The SignBridge web interface achieved consistent 25–30fps video streaming over WebSocket localhost connection with <50ms end-to-end latency from frame capture to browser display. Web Speech API with continuous equal to True and interimResults equal to True provided

real-time speech transcription feedback. GIF playback queue controlled sequences of up to 30 signs without dropping any frame.

F. Networked Translation

TCP socket connection between two machines on the same WiFi network confirmed within 1 second. JSON message latency is less than 5ms. Both translation directions operated simultaneously with no blocking, each machine's four threads ran independently, communicating only final sentences over the network rather than raw video or audio streams.

VI. TESTING AND EVALUATION

A. Unit Testing

Normalization: Cosine similarity >0.99 for same hand at 30cm/60cm/100cm. **Model:** Confusion matrix confirms 32 out of 32 classes correctly predicted. **T5 Gloss:** 11 input combinations tested 7 handled by rule fast path, 4 by T5. **Speech-to-Sign:** 10 spoken phrases tested with correct GIF/fingerspelling display. **Network:** Connection in $<1s$ and JSON latency $<5ms$ on local networks.

B. Integration Scenarios

(1) Sequential A–Z signing—all correct except J. (2) All 6 gesture signs achieved 100% precision and recall. (3) Mixed sentence: H-E-L-L-O + I LOVE YOU + Y-O-U $\rightarrow$ T5 output: 'Hello, I love you.' (4) Speech 'Hello how are you' correct GIF/fingerspelling sequence displayed. (5) Live network mode simultaneous bidirectional translation on two machines over WiFi hotspot confirmed. (6) Web interface live video stream, real-time predictions, and GIF playback all functional in browser.

C. Known Limitations

(1) Sign J: 67% recall due to motion trajectory LSTM-GRU or optical flow needed. (2) T5-small: requires 242MB disk space; rule-based fallback activates when unavailable. (3) Word-level ASL GIFs: limited to available library; most words are fingerspelled. (4) Single signer: model optimized for recording signer; retraining required for others. (5) Web STT: relies on browser's Web Speech API, which requires internet for cloud-based recognition.

VII. FUTURE SCOPE

Mobile Deployment: TensorFlow Lite conversion for iOS/Android edge deployment.

Extended Vocabulary: Transfer learning from WLASL for 500+ sign classes. **LSTM-GRU**

Hybrid: GRU reset gates for improved motion-trajectory sign classification (J, Z). **Multi-**

Signer Support: Fine-tuning with 10–15 sequences per class for new signers. **Ablation**

Study Publication: Quantitative comparison of 258-dim vs 126-dim with/without wrist normalization constitutes a publishable contribution. **Frontend Enhancement:** React.js SPA

with WebRTC for true peer-to-peer browser video calls. **Optical Flow:** Dedicated motion stream for J and Z trajectory classification.

VIII. CONCLUSION

This paper presented a Real-Time Bidirectional Sign Language Translator tracing and addressing the most critical gaps in existing literature which includes the unidirectionality, scale sensitivity, absence of NLP integration, lack of networked peer-to-peer capability, and absence of a browser-based interface.

The key architectural contribution was removing pose and face landmarks and applying wrist-origin scale normalization this solved the domain gap that limited prior approaches to controlled environments. The 126-dim normalized hand feature vector achieves 98.96% accuracy, surpassing all comparable published results, while running at ~15ms inference on commodity CPU hardware.

The T5-small NLP integration enables natural sentence formation from fingerspelled letters, motion gate and confidence gap mechanisms achieve near-zero false positives, the networked live translation mode two machines conducting simultaneous bidirectional ASL communication over TCP with no turn-taking establishes a new baseline for real-time accessible communication systems. The SignBridge web interface makes the system accessible from any browser without installation. Together, these deployed solution help in representing a comprehensive and deployable solution to the sign language communication barricade.

Acknowledgements

The authors thank their project guide, Mrs. Archana Lakhe, for her invaluable guidance and continuous support. Gratitude is extended to the Department of M.Tech (Data Science), MPSTME, SVKM's NMIMS, Mumbai, and the open-source community behind MediaPipe, TensorFlow, HuggingFace Transformers, OpenCV, and Flask-SocketIO.

References

- Bohacek, M. and Hruz, M. (2022). Sign Pose-Based Transformer for Word-Level Sign Language Recognition. In Proceedings of the IEEE WACVW, pp. 182–191.
- De Castro, G. Z., Guerra, R. R. and Guimaraes, F. G. (2023). Automatic Translation of Sign Language with Multi-Stream 3D CNN. Expert Systems with Applications, 215, 119394.
- Dardas, N. H. and Georganas, N. D. (2011). Real-Time Hand Gesture Detection and Recognition Using Bag-of-Features and SVM. IEEE Transactions on Instrumentation and Measurement, 60(11), 3592–3607.
- Farhan, Y. and Madi, A. (2022). Real-Time Dynamic Sign Recognition Using MediaPipe. In Proceedings of IEEE ICECOCS, Fez, Morocco, pp. 1–7.
- Gaikwad, R. and Admuthe, L. (2024). Real-Time Sign Language Recognition of Words and Sentence Generation using MediaPipe and LSTM. In Proceedings of IJCACI 2022, Springer, pp. 287–297.
- Hochreiter, S. and Schmidhuber, J. (1997). Long Short-Term Memory. Neural Computation, 9(8), 1735–1780.
- Huang, J., Chaijaruwanich, J. and Chouvatut, V. (2024). Video-Based Sign Language Recognition with R(2+1)D and LSTM Networks. In Proceedings of IEEE KST, pp. 214–219.
- KhajehPourian, D. and Zarei, A. D. (2024). Sign Language Recognition using Modified Deep Learning Network: CNNSa-LSTM. Scientific Reports, 14.

- Khartheesvar, G., Kumar, M., Yadav, A. K. and Yadav, D. (2024). Automatic Indian Sign Language Recognition using MediaPipe Holistic and LSTM Network. *Multimedia Tools and Applications*, 83, 58329–58348.
- Kothadiya, D. R. et al. (2022). DeepSign: Sign Language Detection and Recognition Using Deep Learning. *Electronics*, 11(11), 1780.
- Kothadiya, D. R. et al. (2023). SIGNFORMER: DeepVision Transformer for Sign Language Recognition. *IEEE Access*, 11, 4730–4739.
- Lugaresi, C. et al. (2019). MediaPipe: A Framework for Perceiving and Processing Reality. In *Proceedings of CVPR Workshop on CV for AR/VR*.
- Mercanoglu, O., Tur, A. O. and Keles, H. (2019). Isolated Sign Language Recognition with Multi-Scale Features using LSTM. In *Proceedings of IEEE SIU, Sivas, Turkey*, pp. 1–4.
- Navendu, P. and Sahula, V. (2024). Word Level Sign Language Recognition using MediaPipe and LSTM-GRU Network. *TechRxiv*.
- Pathan, R. K. et al. (2023). Sign Language Recognition using Fusion of Image and Hand Landmarks. *Scientific Reports*, 13.
- Rajalakshmi, E. et al. (2023). Multi-Semantic Discriminative Feature Learning for Sign Gesture Recognition. *IEEE Access*, 11, 2226–2238.
- Sundar, B. and Bagyammal, T. (2022). American Sign Language Recognition for Alphabets using MediaPipe and LSTM. *Procedia Computer Science*, 215, 642–651.
- Zhang, Y. and Jiang, X. (2024). Recent Advances on Deep Learning for Sign Language Recognition. *Computer Modeling in Engineering and Science*, 139(3), 2399–2450.
- Zhou, Z., Toh, V. W. L. and Lim, E. Y. (2021). SignBERT: A BERT-Based Deep Learning Framework for Continuous SLR. *IEEE Access*, 9, 161669–161682.
- Anonymous (2024). Real-Time Bidirectional Sign Language Translation System using MobileNetV2 and Flutter. *IEEE Xplore*. DOI: 10.1109/11019673.